



CodingCare: AI Code Generation Security Framework for Common Vulnerability Mitigation

Zhiguo Ding*
The Third Research Institute of
Ministry of Public Security
Shanghai, China
dingzhiguo@gass.ac.cn

Songyang Wu
The Third Research Institute of
Ministry of Public Security
Shanghai, China
wusongyang@stars.org.cn

Yilin Zhang
Shanghai University of Engineering
Science
Shanghai, China
miraclelin@sues.edu.cn

Tao Yang
The Third Research Institute of
Ministry of Public Security
Shanghai, China
yangtao@stars.org.cn

Yingna Li
The Third Research Institute of
Ministry of Public Security
Shanghai, China
ziying219@126.com

Zhuhua Li
The Third Research Institute of
Ministry of Public Security
Shanghai, China
lizhuhua310@163.com

Hang Shao
The Third Research Institute of
Ministry of Public Security
Shanghai, China
mir_soh@163.com

Yicong Shi
The Third Research Institute of
Ministry of Public Security
Shanghai, China
shiyicong2018@163.com

Abstract

This article provides a comprehensive review of code generation LLMs (Large Language Models) focusing on security issues and possible solutions to software development workflows. Recent literature suggests that more than 70% of developers are using AI programming assistants in their day-to-day activities. Code generated by AI contains a plethora of serious vulnerabilities such as Cross-site scripting (XSS), SQL injection vulnerabilities, and unsafe credentials which impact the overall security of software systems. Most of the research, thus far, has focused on either the fine-tuning of larger models, working with the methods for optimal prompting, or looking at security evaluations to detect outcomes in coding LLMs, rather than exploring generative models with an overall design of a security framework. To contribute, we develop an overall security framework for code LLMs including prompt libraries, bibliography databases, vulnerability databases, repositories of programming use-cases for several ways through the software development life-cycles, including requirement analysis, code development, code vetting changes, code iteration, and code submission. We developed an experimental system and we will use it to conduct comparative experiments with three code LLMs including Deepseek-coder-7B, Mistral-7B, and Code Llama-7B. The results of this research indicate that our proposed security framework reduces the number of CVEs reported. This research lays a

groundwork for projects that lead to other research issues related to software security LLMs considerations.

CCS Concepts

• **Security and privacy** → Software and application security; Software security engineering.

Keywords

Large language model, AI code generation, Common vulnerability, Security framework

ACM Reference Format:

Zhiguo Ding, Songyang Wu, Yilin Zhang, Tao Yang, Yingna Li, Zhuhua Li, Hang Shao, and Yicong Shi. 2025. CodingCare: AI Code Generation Security Framework for Common Vulnerability Mitigation. In *2025 4th International Conference on Intelligent Systems, Communications and Computer Networks (ISCCN 2025)*, February 21–23, 2025, Nanjing, China. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3732945.3732990>

1 INTRODUCTION

The advent of large language models (LLMs) has led to significant transformation within the space of software development creating a reliance on code generation tools like GitHub Copilot and ChatGPT. Reports in the sector have shown at least 70% of developers are using some form of AI programming assistant, and that is due to the ability of these tools to automate repetitive tasks while speeding up the timelines of projects within development teams [1]. Lastly, reliance on AI-based code generators introduces security concerns. Developers who use AI-based code generators not only produce more insecure code, but they also believe the code generated is secure, creating insecurity within applications developed by aiding software vulnerabilities, especially XSS, SQL injection, and weak authentication vulnerabilities.

*Corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ISCCN 2025, Nanjing, China

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1520-4/2025/02

<https://doi.org/10.1145/3732945.3732990>

Moreover, they can reduce the software development life cycle by some 20-30%, compared to traditional or conventional development models, but in parallel to this, vulnerabilities from the training data and engineering of the tools emerge security risks exist to organizations. For instance, two researchers reported on their analysis of GitHub Copilot and noted that 43.88% of all code strings contained CWEs (Common Weakness Enumerations) and that 70% of all code projects that AI-assistance was contributing to had at least one vulnerability at a level classified medium to high based on understanding of risk of vulnerabilities [2] [3]. Further, vulnerabilities arise from unseen factors originating from insecure codebases at training, and other code vulnerabilities also surface once integrated to interfaces without pre-implementation thought to safeguards or precautions for security.

Current research is focused on increasing software security via modifications or variations of LLM models or prompt engineering techniques in use to identify weaknesses at chosen intervention points. In contrast LLM model behavior through a code generation system, is a leaner and scalable model for development and examination of behaviors, however, investigation for improvements in code security via prompt engineering alone is still needed. Since the industry is looking for a comprehensive solution to engage LLM improvements for different ecosystems while still enhancing developmental practices - a solution is imminent to become a need of the industry.

So we presenting CodingCare which serves as a security framework designed for multiple LLMs generated code. In this paper, our contributions include:

- (1) A five-stage security architecture: Including requirement analysis, code generation, code inspection, code iteration, and code submission.
- (2) Experimental validation: Comparative experiments based on Deepseek-Coder-7B, Mistral-7B, and CodeLlama-7B demonstrate that the CodingCare security framework reduces CVE occurrence rates by at least 50%.

The paper delivers both theoretical and practical backing for additional research which focus on improving security performance of LLMs generated code.

2 RELATED WORKS

2.1 Security Dimensions of Code Generated by LLMs

Approximately 40% of code completions produced by GitHub Copilot displayed security vulnerabilities in high-consequence situations [4]. Studies have shown that models such as Codex and Jurassic-1 are able to fix code in zeroshot context [5], yet Jesse et al. [6] found these models regularly produced simple programming bugs.

At the same times, efforts to improve software security have also been investigated. Tony et al. [7] introduced SVEN which improved CodeGen's generation of secure code by 25%. Nguyen-Duc et al. [8] conducted a security review of Copilot, CodeWhisperer, and ChatGPT and found no major vulnerabilities, yet they noted the HumanEval prioritized functional correctness over spotting security flaws. Studies focused on users yielded mixed results, where Sandoval et al.'s [9] basic task usage of Codex produced no greater

critical security vulnerabilities. In contrast, Torka et al. [10] indicated lower rates of security vulnerabilities among Codex users which were related to library issues and inadequate input sanitization. Chen et al. [11] noted the significance of secure prompting actions during code generation.

2.2 Curated Datasets for Security-Oriented Prompt Engineering

Specialized datasets for using prompts to produce secure code with LLMs have also been built. Majdinasab et al. [12] created a dataset with 89 code-completion prompts based on the CWEs. This was then used by Tony et al. [13] to develop the LLMSecEval dataset, a dataset designed to evaluate security in LLM-generated code. Following this, Siddiq and Santos [14], developed SecurityEval, a dataset of 130 prompts generating 75 vulnerability types mapped to CWEs. Additionally, Meta's Purple Llama CyberSecEval used their ICD (Insecure Code Detector) to identify security problems and then generate code completion tasks based on the ICD findings. These datasets are useful to improve prompts that will improve the security of LLM-generated code.

3 VULNERABILITIES IN GENERATED CODE

Research attention has grown toward the security of code generated by LLMs after findings showed it has vulnerabilities including SQL injection and XSS as well as buffer overflows and so on. LLMs produce security flaws because their training data may encompass insecure code while their understanding of secure coding principles remains insufficient.

CVEs detail publicly available security weaknesses that attackers might exploit to damage software system confidentiality, integrity or availability. A comprehensive understanding of common CVE types enables effective security risk assessment and mitigation especially when dealing with LLM generated code. The following CVE types are relevant to our study.

- Denial of Service (DoS): Attackers achieve service disruption through consuming services or network connections.
- Code Execution: Execution of malicious code on the target system can result in system control and/or data compromise.
- Buffer Overflow: Excess data causes adjacent memory to be overwritten, allowing third-party code to be executed.
- Memory Corruption: Memory is inappropriately accessed or manipulated, causing crashes or vulnerabilities within the application.
- SQL Injection: Malicious SQL code is injected to access or modify data from database.
- XSS: Malicious code, which allowing actions likes session hijacking, is executed via a user's browser.

4 CODINGCARE: AI CODE GENERATION SECURITY FRAMEWORK

The CodingCare framework is used for removing vulnerabilities for AI-generated code and incorporating security checks, which is a multi-stage security framework that during the software development lifecycle. And the framework includes five stages, as

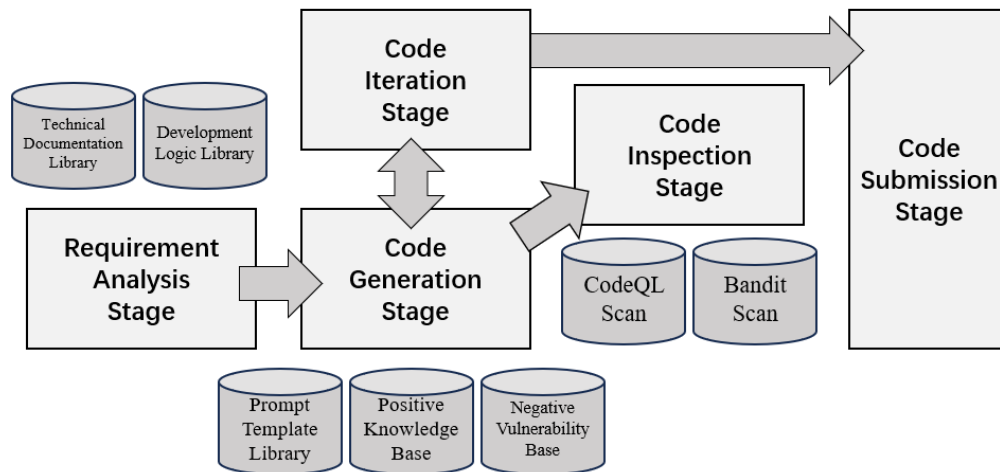


Figure 1: CodingCare: AI Code Generation Security Framework.

illustrated in Figure 1, such as Requirement Analysis, Code Generation, Code Inspection, Code Iteration, and Code Submission. Each stage is used to put a focus on different aspects of security by using combinations of knowledge bases or automation tools to output more secure code.

4.1 Requirement Analysis Stage

The first step of CodingCare process framework is oriented toward turning user requirements that seem to have an undefined meaning into pragmatic functional requirements. The first step is essential for establishing a framework of safe code development as it clarifies the desired outcomes, approaches, and technical specifics for the framework. The framework has components of Technical Documentation Library and Development Logic Library.

In this stage, human developers and the AI programming assistant gain a sort of explicit knowledge about functionality and security objectives. The developers diminish any ambiguity and security risks that the customers may potentially knowingly impose by not having a fully defined specification.

4.1.1 Technical Documentation Library. The Technical Documentation Library is designed to be a one-stop-shop that has complete documentation technical specification documents, API documentation, and best-practice approaches for working with different technologies and libraries that is likely to be generated in the context of the overall project. The Technical Documentation Library content will allow the AI programming assistants to find accepted reliable documentation to interpret and configure library software components and APIs.

4.1.2 Development Logic Library. The Development Logic Library is dedicated to documenting and establishing standards for both functional specifications and business logic that realize the delivery of a software development project. The structured code development framework will allow the specification of input requirements, output results and approach unique order processing functions at each functional phase. The Development Logic Library employs the

requirements needed to format an outcome for complex provider search functionality that lives in the context of an e-commerce system. As an example, the input requirements may consist of parameters like keywords, price range and the output requirements denote a formatted display into a product listing display, while unique order rules to execute the search algorithm could be included in the library.

4.2 Code Generation Stage

In line with the framework design, the Code Generation Stage refers to the actual generation of code by the AI programming assistant. This stage is reviewed to encourage secure and robust code generation by leveraging multiple software security library nodes in the framework. Collectively, the software security library architecture provides a Prompt Template Library, a Positive Knowledge Base, and a Negative Vulnerability Library. Through the prompted use of the three software security library nodes, CodingCare's Code Generation Stage will support the AI programming assistant to build code that is more secure at its foundation.

4.2.1 Prompt Template Library. The Prompt Template Library represents a set of pre-defined prompt templates to help target security considerations into the code generation in a more deliberate way. The prompt templates are created from best practices of Prompt Engineering and security guidelines for a prompt approach to leverage direct control over the LLM's output. Utilizing methods such as RCI (Recursive Criticism and Improvement) and CoT (Chain of Thought) methods or some similar methods, these templates can be modified to aid the LLM to incrementally source security towards code generation methods. For example, prompts may be developed to specifically ask for following secure coding practices and property specifications, asking for an explicit source of input-level validation, and requesting the LLM to consider and list possible security implications to consider when developing the code.

4.2.2 Positive Knowledge Database. The Positive Knowledge Base serves as a repository of quality coding practices, forwarding-level

documentation of expected coding practices for relevant code packages, and organizational based coding guidance. This would include coding style guidance, recommendations on naming conventions, helpful indenting if necessary, and how to comment instructions and specifically, which interests. By using the Positive Knowledge Base, the LLM will know how to configure necessary packages, and how to safely use the means code packages, while also adhering to code guidelines, if there are other established coding norms and principles to use. Secure code, functional code, and maintainable code is implicated thinking is writing of the code to prevent the systems use as inappropriate means and uses.

4.2.3 Negative Vulnerability Database. The Negative Vulnerability Database represents a curated CVE database. This database provides detailed background information on CVEs, including root cause of the issue, impact to the system or user, and then recommendations on how to incrementally address the root cause, and thus limit risk. The coverage of the Negative Vulnerability Database includes insecure coding practices such as dangerous code function calls and improper use of uninitialized variables. This framework utilizes the Negative Vulnerability Database to direct the LLM to understand secure code and deter and prevent codes that have security weaknesses based on existing vulnerabilities.

4.3 Code Inspection Stage

Once generated Code, the Code Inspection Stage follows because the Code Inspection Stage will automatically identify possible security defects in the generated Code. In this stage, we will have automated static analysis tools such as CodeQL Scan and Bandit Scan to perform and autocode security scans.

Using CodeQL and Bandit in the Code Inspection Stage provides for a thorough automated process of inspecting code security defects that can leverage each other. The Code iteration stage monitors the scan results for the ability to fix defects in out code that were discovered in previous stages. These tools were selected because they are evidenced by industry usage and satisfaction, and they can be considered effective systems for static code analysis, but with Python's reputation as being code is what these code scan tools will closely match which is a component of AI generated code.

4.3.1 CodeQL Scan. GitHub's CodeQL Scan is a semantic code analysis tool that detects code vulnerabilities in a wide range of programming languages, is highly effective tested in both code and program [15]. CodeQL, employs a robust semantic contextual understanding of code, it detects complex code vulnerabilities.

4.3.2 Bandit Scan. Bandit Scan is a semantic code analysis tool similar to CodeQL, it is designed to detect vulnerabilities in Python code only [16]. Bandit is also effective in its ability to traverse code files, build abstract syntax trees and report possible security vulnerabilities categorized by CWEs.

4.4 Code Iteration Stage

The Code Iteration Stage of the framework will expect human developers to engage with the framework, communicate, and interpret the code and code adaptability deviation queries associated with code risks. This stage expects human developers to read, manage, and modify code files and manage the scope for the project

based on their documentation to revise code. In Code Iteration, human developers can interpret their own application of CodeQL and Bandit.

In order complete documentation practices and keep a history record of code documents, human will need to interface and practice reason/development documentation reasons regarding the code modifications and updates. In Code Iteration, human developers have the opportunity for ongoing improved code base in a more organized way that is 'safe' while also maintaining functionality and efficiency. And beyond engagement in the documentation and overall enjoy managing the quality and security of their code related to their interests and project, their actual engagement includes activities to help document and interpret readings of the findings from automated scans of code, engage risks patches to vulnerabilities, and consider their existing knowledge related to good code security and overall code quality.

4.5 Code Submission Stage

The fifth phase of the CodingCare framework which the iterative refinements of code has reinforced improvements of enhanced good secure code development occurs, the phase occurs, pointing to the point the code outputs are ready for post completion, submission to code repository. Submitting code marks the changing point in the overall lifecycle of software development through multiple phases for code that will transition into either testing integration, or deployment.

5 EXPERIMENTAL VALIDATION

5.1 Experimental System

We developed an experimental configuration that utilized the CodingCare framework, which included Gitlab, VSCode, AnythingLLM, and three publicly available code LLMs (Large Language Models): Deepseek-coder-7B, Mistral-7B, and Code Llama-7B.

Gitlab is a web-based DevOps platform, which represents a complete suite of tools for the software development life cycle. Gitlab acts as a code repository to hold and manage the project code that is generated by code LLMs, and is intended to replicate a real-world software development workflow. VSCode, developed by Microsoft, is a small footprint, free, open-source code editor that supports the primary integrated development environment (IDE). Developers can use VSCode to write and review code and connect to the code LLMs through the AnythingLLM extension of VSCode to receive code from code LLMs they generate and review the code. AnythingLLM is an open-source platform that connects multiple code LLMs to increase user interaction with code LLMs. More specifically, AnythingLLM acts as a bridge from the VSCode platform to the code LLMs back to the VSCode platform. Additionally, AnythingLLM manages the prompts and integrates the CodingCare prompt library and knowledge base for safer and higher quality prompts for the code LLM.

Deepseek-coder-7B is an open-source code LLM developed by the DeepSeek Company, with 7 billion parameters. Deepseek-coder-7B is one of the coding LLMs being utilized in this configuration and will be responsible for generating the project code for the experiments. Mistral-7B is a state-of-the-art open-source language model from Mistral AI, hosting 7 billion parameters, and serves as

Table 1: CVE performance of 5 experimental web projects

Experimental Web Project	CVE Average Count without CodingCare	CVE Types without CodingCare	CVE Average Count With CodingCare	CVE Types with CodingCare
E-commerce Platform	3	SQL Injection XSS Sensitive Data Exposure	1	XSS
Community Forum	4	SQL Injection Stored XSS Sensitive Data Exposure Clickjacking	1	Stored XSS
Personal Blog	2	Insecure Direct Object Reference Sensitive Data Exposure	0	-
News Portal	1	SQL Injection	0	-
Backend Admin System	2	Weak Passwords File Upload Vulnerability	1	File Upload Vulnerability

another code LLM being utilized in the project. Code Llama-7B, which was released in 2023 by Meta, is another code LLM we added simply because it is a robust representation of the public code LLMs available to the computing community.

5.2 Experimental Design

To evaluate the prognosis outcomes of the CodingCare framework we used five standard web application development projects which included e-commerce, a community forum, a personal blog, a news portal, and a backend management system. Each of the three LLM models (Deepseek-coder-7B, Mistral-7B, and Code Llama-7B) were used to build these projects because they built different components of these applications.

- E-commerce Platform (Product Display Module): Specific features that users could build included product browsing and searching which are typically vulnerable to SQL injection and information leakage.
- Community Forum: Features could also include functionality for posts and replies which typically suffer from XSS attacks.
- Personal Blog: The user could also build features for user registration, article publishing, commenting, etc which are vulnerable to XSS and SQL injection vulnerabilities.
- News portal: Users would also build functionality that included information display and user interaction which is typical of XSS and CSRF risks.
- Backend Admin System (simplified): A user could build a system that required user authentication and data management functionality which could be vulnerable because of insecure authentication and authorization.

We brought together subject matter expert human programmers to review the generated code in hopes of detecting CVEs in each of the projects. The review of the code focused on attacks such as XSS including SQL injection vulnerabilities and insecure authentication vulnerabilities. The audit of each code base was result was documented and each code base was reviewed comparatively with each of the models.

6 EXPERIMENTAL RESULTS AND ANALYSIS

The experimental results (Table 1) demonstrate that the number of security vulnerabilities in project code generated using the CodingCare is significantly lower than when code is generated directly using LLMs without the framework.

During the security audit of the E-commerce project developed without CodingCare, three vulnerabilities were identified including SQL Injection, XSS, and Sensitive Data Exposure. In the security audit of the Community Forum project, four vulnerabilities were identified including SQL Injection, Stored XSS, Sensitive Data Exposure, and Clickjacking. The Personal Blog project contained at least two CVEs: Insecure Direct Object References and Sensitive Data Exposure. The News Portal project demonstrated SQL Injection, while the Backend Admin project had two types of vulnerabilities including Weak Passwords and File Upload issues.

When applied to web application projects, the CodingCare framework showed a substantial decrease in CVEs: the E-commerce project saw a 67% reduction, the Community Forum project had a 75% reduction, the Personal Blog and News Portal projects achieved a 100% reduction, while the Backend Admin project exhibited a 50% reduction. Overall CVE reductions were substantive for E-commerce and Community Forum while Personal Blog and News Portal had zero vulnerabilities. However, the Backend Admin project showed one remaining vulnerability which derived from File Upload issues.

Though additional security was implemented, the generated code for projects still contained vulnerabilities including XSS and File Upload vulnerabilities. Code generation LLMs did not have sufficient contextual understanding of each unique project's goals, the system's input and output parameters, and the necessary user input and upload patterns of behavior which will impact the overall security of generated code. The holistic security of generated code relies on these aspects and even with the CodingCare Framework fine-tuning additional work is required in some areas to resolve higher edge cases more effectively.

7 CONCLUSION

The findings indicate that the security framework, CodingCare, discussed in this study has the potential to significantly decrease the number of CVEs in the development of web applications while creating better software product. This evidence also provides theoretical and practical evidence for research related to improving the security of LLM generated code.

Acknowledgments

The authors would like to acknowledge their colleges and researchers who have contributed to this paper. This research was partially supported by Project Fund for Young Scientists Research Class, Grant No. C23253.

References

- [1] Coutinho, M., Marques, L., Santos, A., Dahia, M., França, C., & de Souza Santos, R. (2024, July). The role of generative ai in software development productivity: A pilot case study. In *Proceedings of the 1st ACM International Conference on AI-Powered Software* (pp. 131-138).
- [2] Prakash, M. (2024). Role of Generative AI tools (GAITs) in Software Development Life Cycle (SDLC)-Waterfall Model (Doctoral dissertation, Massachusetts Institute of Technology).
- [3] Moroz, E. A., Grizkevich, V. O., & Novozhilov, I. M. (2022, January). The Potential of Artificial Intelligence as a Method of Software Developer's Productivity Improvement. In *2022 Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus)* (pp. 386-390). IEEE.
- [4] Mohsin, A., Janicke, H., Wood, A., Sarker, I. H., Maglaras, L., & Janjua, N. (2024). Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs. *arXiv preprint arXiv:2406.12513*.
- [5] Pearce, H., Tan, B., Ahmad, B., Karri, R., & Dolan-Gavitt, B. (2023, May). Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)* (pp. 2339-2356). IEEE.
- [6] Jesse, K., Ahmed, T., Devanbu, P. T., & Morgan, E. (2023, May). Large language models and simple, stupid bugs. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)* (pp. 563-575). IEEE.
- [7] Tony, C., Ferreyra, N. E. D., Mutas, M., Dhiff, S., & Scandariato, R. (2024). Prompting techniques for secure code generation: A systematic investigation. *arXiv preprint arXiv:2407.07064*.
- [8] Nguyen-Duc, A., Cabrero-Daniel, B., Przybylek, A., Arora, C., Khanna, D., Herda, T., ... & Abrahamsson, P. (2023). Generative Artificial Intelligence for Software Engineering—A Research Agenda. *arXiv preprint arXiv:2310.18648*.
- [9] Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., & Dolan-Gavitt, B. (2023). Lost at c: A user study on the security implications of large language model code assistants. In *32nd USENIX Security Symposium (USENIX Security 23)* (pp. 2205-2222).
- [10] Torka, S., & Albayrak, S. (2024). Optimizing AI-Assisted Code Generation. *arXiv preprint arXiv:2412.10953*.
- [11] Chen, Y., Sun, W., Fang, C., Chen, Z., Ge, Y., Han, T., ... & Xu, B. (2024). Security of Language Models for Code: A Systematic Literature Review. *arXiv preprint arXiv:2410.15631*.
- [12] Majdinasab, V., Bishop, M. J., Rasheed, S., Moradidakhel, A., Tahir, A., & Khomh, F. (2024, March). Assessing the Security of GitHub Copilot's Generated Code-A Targeted Replication Study. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (pp. 435-444). IEEE.
- [13] Tony, C., Mutas, M., Ferreyra, N. E. D., & Scandariato, R. (2023, May). Llmseceval: A dataset of natural language prompts for security evaluations. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)* (pp. 588-592). IEEE.
- [14] Siddiq, M. L., & Santos, J. C. (2022, November). SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques. In *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security* (pp. 29-33).
- [15] Aanby, J. L., Evensen, D., & Myklebust, V. (2024). *Bridging the Gap Between Software Security and Development using CodeQL* (Bachelor's thesis, NTNU).
- [16] Ruohonen, J., Hjerpe, K., & Rindell, K. (2021, December). A large-scale security-oriented static analysis of python packages in PyPI. In *2021 18th International Conference on Privacy, Security and Trust (PST)* (pp. 1-10). IEEE.